

C LLM Prompts

In this section, we present key prompts used for creating TOD-ProcBench and evaluating LLMs for each task.

Instruction Generation LLM System Prompt (Part 1)

Conversation

You are given a set of conversations between a customer and an agent. In all of the conversations the customer has a specific intent from the interaction with the agent. You are asked to read all the conversations in `<conversations>` tags for the specified intent in `<intent>` tag and generate a policy that the agent has followed to complete all the conversations in `<conversations>` tag.

hint: your generated policy should contain only one section in `<what_to_do>` tags and one `<what_to_say>` tags.

Here is a template for the policy that you are asked to generate. Each template has to have two sections:

1. What to do in `<What_to_do>` tags which shows some conditional statements and the steps to accomplish the task. You can specify each condition with 'if' and then have the condition and what needs to be done for that condition. Try to provide all details as an example directly mention different membership levels and the return dates for each level accordingly.

2. What to say in `<What_to_say>` tags which shows what are the important utterances that the agent has to say to accomplish all the defined steps in the what to do part.

{continued in Table 16}

Table 15: LLM system prompt (Part 1) to generate an instruction document given the set of conversations with similar user intents.

Instruction Generation LLM System Prompt (Part 2)

{continued from Table 15}

<example>

Here is an example of a policy for the following conversation in <example_conv> tag:

<example_conv>

"agent: hello, how are you today? how can i help you?",

"customer: i did a return but i don't want to get my refund as a gift card.",

"agent: can i please get your full name and account id?",

"customer: my name is [name1] but i'm not sure about the account id",

"agent: you are looking to get check instead of gift card?",

"customer: yes",

"action: account has been pulled up for [name1]",

"agent: what is your membership level",

"customer: gold",

"action: membership level of has been noted.",

"agent: since your are a gold member we can refund you via the check. Do you want to use your existing bank account?",

"customer: yes, thanks",

"action": changes have been entered.

"agent: no problem! is there anything else i can help you with, sanya?",

"customer: that will be all",

"agent: perfect. you have a nice day. goodbye!"

</example_conv>

Policy for this conversation is:

<What_to_do>

If customer wants to get check instead of gift card:

-If membership level is Gold or Silver:

- Allow the request
- Ask for the account info if you do not have
- Process the request

-If membership level is guest or bronze:

- if the refund has been initiated in 30 days:
 - Allow the request and ask for the account info and process it
- if not:
 - Inform customer request cannot be processed

</What_to_do>

<What_to_say>

- Ask for customer's full name or account ID

- Check membership level

- Check refund initiated date based on membership level

- If the changing request is eligible:

- Ask customer's bank account info if it does not exist.
- Ask if customer wants to use existing bank account it if exists.

-If the request is not eligible:

- Politely inform customer request cannot be processed and explain reasoning
- Apologize for inconvenience
- Offer to help with anything else

</What_to_say>

</example>

Table 16: LLM system prompt (Part 2) to generate an instruction document given the set of conversations with similar user intents.

Instruction Generation LLM User Prompt

Conversation

Here is the intent and all the conversations. Read all of them carefully and generate the policy accordingly.

<intent> *intent* </intent>

<conversations>

convs

</conversations>

The best policy for the given conversations is:

Table 17: LLM user prompt to generate an instruction document given the set of conversations with similar user intents.

Complex Instruction Format Conversion LLM Prompt

Convert nested conditional statements into two formats which should represent the same meaning in different formats:

1. A flattened text format where each line starts with a condition and lists actions to take
2. A flattened JSON format with an array of objects, each containing "conditions" and "actions" arrays

GENERAL INSTRUCTIONS:

- Identify all decision points and actions in the nested structure
- The entries for conditions and actions in each format must use the same natural language from the original input.
- Maintain all the original logic and relationships between conditions and actions
- Consecutive entries in the flattened formats should NOT have the exact same list for conditions. This effectively means that all condition-action pairs that are neighbors must have different lists for conditions. If the same list is used (same entries in the list or same empty list which specifically denotes in all cases) for conditions in pairs that are consecutive, then combine all such consecutive pairs to only have one unique list of conditions and all corresponding actions in the same original sequence.
- The expectation is that a user of these flattened instructions will read them in order from top-down, first to last, evaluating each condition and if true executing the associated action(s). As such, your flattening must be recursive and preserve the same order of actions executed if the user were to follow the original SOP.
- Process each section separately if multiple sections are provided

INSTRUCTIONS for flattened text format:

- Each line should start with "If [condition]:" followed by indented actions
- When a condition depends on previous conditions, use "AND" to combine them (e.g., "If condition1 AND condition2:")
- For actions that apply in all cases, use "In all cases:" as the condition

INSTRUCTIONS for flattened json format:

- Represent each condition-action pair as "conditions": [...], "actions": [...]
- For actions that apply in all cases, use an empty list [] as the condition

You might find it easier to generate the flattened text format first and then generate the equivalent flattened json format. The motivation here is to flatten the nested hierarchy of conditional statements so that one can simply go down the list from the first line in a flattened format to the last line in a flattened format and only execute actions corresponding to conditional statements that are true.

Please convert this into both flattened formats and provide your output.

Input:

json_data

Provide a single dictionary so that `json.loads()` can parse your perfectly formatted response. Your final response should be a json with the following keys (note that the first 3 keys are the same as original input and the values should be preserved as well):

"intent" "what_to_do", "what_to_say", "flattened_text_what_to_do", "flattened_text_what_to_say",
"json_what_to_do", "json_what_to_say",

Table 18: LLM prompt to convert complex instruction format from f_1 to f_2 and f_3 .